

System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models

Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang,
Reza Yazdani Aminabadi, Shuaiwen Leon Song, Samyam Rajbhandari, Yuxiong He
Microsoft Inc, Redmond, WA, USA

I. INTRODUCTION

Long sequences are ubiquitous in NLP tasks such as document summarization, machine translation, and dialogue modeling [1], [2], [3], [4], [5], [6], [7], [8], [9]. Traditional approaches to parallelism, including data parallelism [10], [11], [12], tensor [13] and pipeline parallelism [14], [15], [16] struggle to handle sequences that span thousands or even millions of tokens.

In this paper, we introduce Ulysses, a novel, simple, portable, and effective methodology for enabling highly efficient and scalable LLM training with extremely long sequence lengths. Ulysses partitions individual samples along the sequence dimension among participating GPUs. Then right before the attention computation, it employs all-to-all communication collective on the partitioned queries, keys and values such that each GPU receives the full sequence but only for a non-overlapping subset of the attention heads. This allows the participating GPUs to compute attention for different attention heads in parallel. Finally, Ulysses employs another all-to-all to gather the results along the attention heads while re-partitioning along the sequence dimension.

Theoretical communication analysis shows that whereas other methods incur communication overhead as sequence length increases, Ulysses maintains constant communication volume when sequence length and compute devices are increased proportionally. Furthermore, experimental evaluations show that Ulysses scales to more than 1 million context length and trains 2.5x faster with 4x longer sequence length than the existing method SOTA baseline.

II. ULYSSES

A. Core System Design

Figure 1 shows the core design of Ulysses. As with the known transformer architecture, the design consists of input sequences of size N partitioned across P available devices. Each local N/P partition is projected into queries (Q), keys (K) and values (V) embeddings. Next, (QKV) embeddings are gathered into global QKV through highly optimized all-to-all collectives between participating compute devices. Sequel to all-to-all collective is the attention computation. After the attention computation, another all-to-all collective transforms output context tensor of attention computation to sequence (N/P) parallel for subsequent operators in the remaining modules of transformer layer block. Essentially, the computation pattern

of Ulysses transitions from sequence parallelism to head parallelism and then back to sequence parallelism. Efficient all-to-all communication is crucial for transitioning between these two modes of parallelism.

B. Communication Analysis

What distinguishes Ulysses from the other existing long-sequence approaches is our much smaller aggregate communication volume and overall better scalability with increasing degree of sequence parallelism compared to existing solutions.

On modern clusters with intra-node NVSwitch interconnect and inter-node fat tree IB topology, the communication volume transmitted per link for an all-to-all for aggregate message of size M over P GPUs is M/P . For a transformer model with hidden size d , sequence length of N , and parallelism degree of P , Ulysses performs all-to-all for the QKV projections with an aggregate message size of $3Nd$ before the attention computation, and another all-to-all for output context projection with a size Nd for each transformer layer. Therefore, Ulysses incurs an aggregate communication volume per link of $4Nd/P$ (or with the complexity of $O(N/P)$). Note that this communication volume is constant when both N and P are increased proportionally.

In contrast, the existing approaches like Megatron-LM [17] and ColAI [18] sequence parallelism incur communication volume that increases linearly with N regardless of P , resulting in the communication complexity of $O(N)$.

C. Additional Benefits

1) *Support for Large Models*: While Ulysses reduces the activation memory when training with longer sequences, it does not impact the memory consumed by the model states. Therefore, to support large sequence length training with a large language model, Ulysses is integrated with ZeRO Redundancy Optimizer (ZeRO) [11], [12]. We extend ZeRO partitioning to combination of data parallel and sequence parallel ranks. ZeRO support allows for huge memory savings in both sequence and data dimensions and enables scaling not just to large sequence lengths but also to large models.

2) *Generality*: Ulysses implementation of distributed attention module is general enough to support any attention: e.g., self-attention, cross-attention, causal attention in both their dense and sparse counterparts [19], [20], [21], [4], and their various optimized kernels that support long-sequence at local attention level such as different versions of FlashAttention [22], [3]. The generality property of Ulysses stems from the

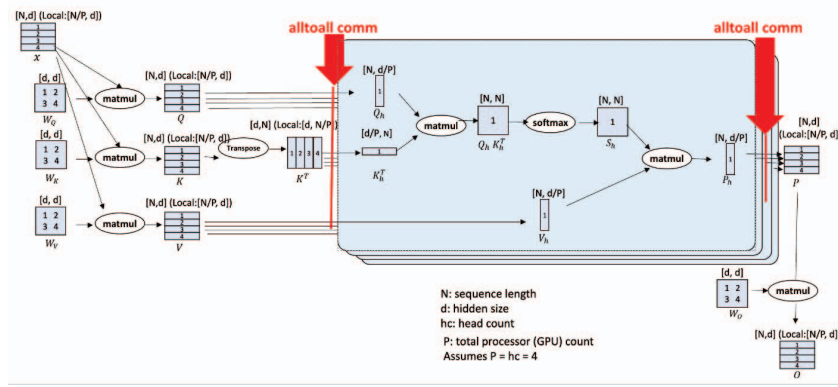


Fig. 1: DeepSpeed Ulysses core design

modular nature of its core design: an attention-centric sequence parallelism design. Attention computation can be replaced with any type of attention mechanisms

III. EVALUATION

We evaluate Ulysses on GPT [23], a foundation model for many NLP tasks. Evaluations reported here are two folds: i) sequence length scalability, ii) Throughput comparison with existing sequence parallelism methods.

A. Sequence Length Scalability

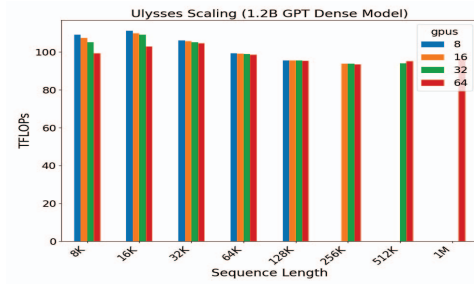
The first set of experiments is strong scaling of sequence length up to 1 million tokens on 1.2 billion parameter GPT model (GPT-1.2B). Results of this evaluation are shown in Figure 2a. Ulysses allows increasing sequence length linearly with the number of GPUs and sequence length scales linearly relative to and maintains similar computation throughput across different sequence length.

B. Throughput Comparison with existing Methods

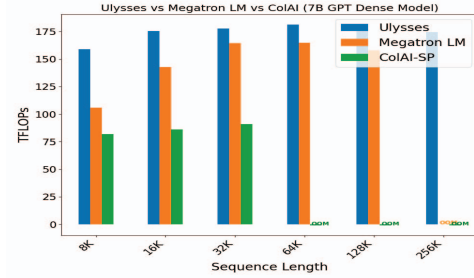
Next, we evaluated Ulysses on 7 billion (7B) parameter GPT dense attention models and compared against Megatron-LM's sequence parallelism (Megatron LM) and Colossal AI sequence parallelism (ColAI-SP) on 32 A100 GPUs. The results of these evaluations are shown in Figures 2b.

We compared Ulysses with Megatron-LM and ColAI-SP for GPT-7B models running various sequence lengths. We chose the sequence parallelism degree and micro-batch size that produced the best performance (measured as TFLOPs) for the three methods, this we call optimal (batch size-sequence length) configurations.

Figures 2b shows that Ulysses consistently outperforms Megatron-LM and ColAI-SP for the sequence length that can be run with them. In addition, Ulysses can run longer sequence than the two existing methods. Ulysses performance advantages are two folds: (1) Ulysses in combination with ZeRO-3 parameter sharding across both data and sequence parallel groups fits more samples than Megatron-LM and ColAI-SP because of the memory optimization leading to higher throughput (2) Ulysses benefits from efficient all-to-all



(a) Ulysses strong scalability evaluation of GPT-1.2B at different sequence length and GPU counts



(b) Evaluation of Ulysses vs Megatron LM vs ColAI-SP on GPT-7B parameter model

Fig. 2: Experimental evaluation

communication relative to all-gather reduce-scatter and ring-style P2P communication as applied in Megatron-LM and ColAI-SP sequence parallelism.

IV. CONCLUSION

DeepSpeed Ulysses represents a significant leap forward in the field of large-scale language modeling. By prioritizing sequence parallelism and optimizing communication, it empowers researchers and practitioners to tackle complex NLP tasks with longer context. As LLMs continue to evolve, DeepSpeed Ulysses will play a pivotal role in shaping the future of AI-driven natural language understanding and other emerging long context applications [24], [25].

REFERENCES

- [1] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler, "Long range arena: A benchmark for efficient transformers," 2020.
- [2] OpenAI, "Gpt-4 technical report," 2023.
- [3] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023.
- [4] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020.
- [5] W. Kryściński, N. Rajani, D. Agarwal, C. Xiong, and D. Radev, "Booksum: A collection of datasets for long-form narrative summarization," 2022.
- [6] MosaicML, "Introducing mpt-7b: A new standard for open-source, commercially usable llms," <https://www.mosaicml.com/blog/mpt-7b>, 2023.
- [7] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K. A. Sankararaman, B. Oguz, M. Khabsa, H. Fang, Y. Mehdad, S. Narang, K. Malik, A. Fan, S. Bhosale, S. Edunov, M. Lewis, S. Wang, and H. Ma, "Effective long-context scaling of foundation models," 2023.
- [8] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "Yarn: Efficient context window extension of large language models," 2023.
- [9] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," 2023.
- [10] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang *et al.*, "Large scale distributed deep networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [11] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–16.
- [12] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21, 2021.
- [13] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2019.
- [14] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. Devanur, G. Granger, P. Gibbons, and M. Zaharia, "Pipedream: Generalized pipeline parallelism for dnn training," in *ACM Symposium on Operating Systems Principles (SOSP 2019)*, October 2019.
- [15] Y. Huang, Y. Cheng, D. Chen, H. Lee, J. Ngiam, Q. V. Le, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," *ArXiv*, vol. abs/1811.06965, 2018.
- [16] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, "Memory-efficient pipeline-parallel dnn training," in *International Conference on Machine Learning*. PMLR, 2021, pp. 7937–7947.
- [17] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoenybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," 2022.
- [18] S. Li, F. Xue, C. Baranwal, Y. Li, and Y. You, "Sequence parallelism: Long sequence training from system perspective," 2022.
- [19] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *CoRR*, vol. abs/1904.10509, 2019.
- [20] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Kane, T. Sarlós, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. J. Colwell, and A. Weller, "Rethinking attention with performers," *CoRR*, vol. abs/2009.14794, 2020. [Online]. Available: <https://arxiv.org/abs/2009.14794>
- [21] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," 2021.
- [22] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," 2022.
- [23] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," 2019.
- [24] T. Nguyen, J. Brandstetter, A. Kapoor, J. K. Gupta, and A. Grover, "Climax: A foundation model for weather and climate," 2023.
- [25] M. Zvyagin, A. Brace, K. Hippe, Y. Deng, B. Zhang, C. O. Bohorquez, A. Clyde, B. Kale, D. Perez-Rivera, H. Ma *et al.*, "Genslms: Genome-scale language models reveal sars-cov-2 evolutionary dynamics," *bioRxiv*, pp. 2022–10, 2022.